

# GCP Parity Gap Analysis — Vyomi Simulator

Generated 2026-05-26. Scope: all GCP services, four dimensions — **Console**, **API (REST)**, **Java/Go SDK**, **Utilities (gcloud/goutil + data-plane CLIs)**.

## 0. How to read this report

Parity is rated per dimension:

- **Full / Good** — works end-to-end, close to real GCP.
- **Partial** — core path works; notable gaps.
- **Minimal / Theoretical** — exists but shallow, or plausible-but-unverified.
- **Blocked / None** — architecturally can't work as-is.

**Critical methodology note on what is *verified* vs *theoretical*:**

- The conformance harness ( `tests/conformance/run_conformance.py` ) exercises GCP **only via raw HTTP/JSON** ( `urllib` ), so it verifies the **REST contract shape** — not real client libraries. AWS, by contrast, is tested with **real boto3** clients. So **every GCP "SDK/CLI works" claim below is theoretical unless stated otherwise**.
- The simulator speaks **HTTP/REST JSON only** — **there is no gRPC**. This is the single most important fact for SDK parity (see §3).

## 1. Executive scorecard

| Service         | Console | API (REST) | Java SDK              | Go SDK                           | Utilities         | Data-plane fidelity                   |
|-----------------|---------|------------|-----------------------|----------------------------------|-------------------|---------------------------------------|
| Compute Engine  | ● 85%   | ● 70%      | ● REST-ready          | ● verified (list)                | ● translator-only | ● real LXD VMs + IPs                  |
| Cloud Storage   | ● 80%   | ● 70%      | ● verified            | ● verified                       | ● gcloud verified | ● real bytes (fake-gcs-server)        |
| Cloud SQL       | ● 80%   | ● 60%      | ● data / ● admin      | ● data / ● admin verified (list) | ● psql/mysql work | ● real Postgres/MySQL                 |
| Pub/Sub         | ● 85%   | ● 85%      | ● verified (emulator) | ● verified (emulator)            | ● translator-only | ● emulator (console↔SDK shared state) |
| Firestore       | ● 80%   | ● 70%      | ● verified (emulator) | ● verified (emulator)            | ● translator-only | ● emulator (console↔SDK shared state) |
| Cloud Functions | ● 75%   | ● 65%      | ● REST-ready          | ● verified (list)                | ● translator-only | ● real code exec (subprocess)         |
| API Gateway     | ● 65%   | ● 50%      | ● untested            | ● untested                       | ● translator-only | ● real request routing                |
| VPC Network     | ● 65%   | ● 55%      | ● untested            | ● untested                       | ● translator-only | ● real iptables enforcement           |
| IAM             | ● 60%   | ● 55%      | ● REST-ready          | ● verified (list)                | ● translator-only | ● real PDP (opt-in)                   |

**Route surface:** ~261 routes total — **119 native Google-style** ( `/compute/v1/...` , `/storage/v1/...` , `/sql/v1beta4/...` , `/v1/projects/...` , `/firestore/v1/...` ) + 142 `/api/gcp/...` console aliases.

### Verified results — P0 progress (2026-05-26)

Real client libraries, run against the appliance, now **pass** for Cloud Storage:

- **Java google-cloud-storage 6/6** (buckets insert/list/delete, objects insert/get-byte-exact/delete) via `StorageOptions.setHost(...) + NoCredentials`.
- **Go cloud.google.com/go/storage 5/5** via `STORAGE_EMULATOR_HOST`.
- **gcloud storage CLI 6/6** (create/list/cp/cat/rm/delete) via `CLOUDSDK_API_ENDPOINT_OVERRIDES_STORAGE + auth/disable_credentials`.

Getting there required fixing **5 cross-cutting bugs the raw-HTTP harness never caught** — all benefit *every* Google client, not just Storage:

1. **gzip request bodies** — Google clients gzip JSON request bodies; `request.json()` 500'd on the gzip magic bytes. Added a global gzip/deflate request-decompression ASGI middleware.
2. **int64 overflow** — `_gcp_compute_numeric_id` (used for `generation`, `projectNumber`, `ids`) could exceed signed int64; Apiary parses these as `long` → intermittent `IllegalArgumentException`. Masked to 63 bits.
3. **LockedTime: ""** — Apiary parses it as a timestamp; empty string threw. Dropped when empty.
4. **/download/storage/v1/...** — Java media-download path wasn't routed (fell through to AWS S3). Added route + marked GCP-native.
5. **GCS XML download GET /{bucket}/{object}** — Go reads via the XML path, which collided with the S3 path-style handler. Added a GCS-byte-store fallback in `s3_get_object`. (AWS S3 verified non-regressed.)

Harness lives in `tests/conformance/gcp-sdk-java/` (native Maven on host), `tests/conformance/gcp-sdk-go/` (dockerized `golang`, `--network host`), `tests/conformance/gcp-cli-smoke.sh` (dockerized `cloud-sdk`).

**P1 progress — gRPC services unblocked:** the official Google Pub/Sub + Firestore emulators are deployed as compose services (`cloudlearn-pubsub:8085`, `cloudlearn-firestore:8080`) and the simulator is wired with `PUBSUB_EMULATOR_HOST` / `FIRESTORE_EMULATOR_HOST`. Verified: **real Go cloud.google.com/go/pubsub 5/5** (CreateTopic, CreateSubscription, Publish, streaming Receive+Ack, delete) via `PUBSUB_EMULATOR_HOST` — the previously gRPC-blocked SDK now works. Firestore is unblocked by the identical mechanism. Remaining P1: make the simulator's console/REST handlers **delegate** to the emulators so the console and the SDKs read one shared state (needs `google-cloud-pubsub/grpc` added to the simulator image). Harness: `tests/conformance/gcp-sdk-go-pubsub/`.

**P0 extended — admin/REST services verified:** the official Google REST clients (`google.golang.org/api`) now list + parse the simulator's responses for **Compute (instances + networks), Cloud SQL Admin (instances), Cloud Functions, and IAM (service accounts)** — 5/5 (harness `tests/conformance/gcp-sdk-go-admin/`). One more shape bug fixed: Cloud Functions `timeout` is now a protobuf **Duration string** ("60s") not an int (`_gcp_duration_str`). Caveat: this verifies the **read/list path; create with long-running-operation polling** (Compute/SQL/Functions return an `Operation` in real GCP, the sim returns the resource directly) is the remaining gap → tracked under P3 (LRO fidelity). The endpoint-override convention differs per client: Compute base = `.../compute/v1/`, but Cloud SQL/Functions/IAM base = `.../` (the version is in the method path).

**P3 progress — LRO (long-running operation) fidelity:** create/delete on Compute/SQL/Functions now return a proper `Operation` that clients poll to done, closing the `create+Wait()` gap (and what Terraform needs):

- **Cloud SQL:** `instances.insert/delete/restart` return a `sql#operation`; added `operations.get`. Verified: real Go `sqladmin 4/4` (Insert→Operation → poll `operations.get` → DONE → `instances.get` → Delete→Operation).
- **Cloud Functions:** `functions.create` returns a `google.longrunning.Operation` (`done:true`, `response` = the function); added `GET /v1/operations/{op}`. Verified: real Go `cloudfunctions/v1 3/3` (Create→Operation → poll → get). Also normalized the function record to key by short id (clients pass the full resource name) and emitted `timeout` as a Duration string.
- **Compute:** already returned a `compute#operation` (insert→PENDING, `zoneOperations.get`→DONE) — unchanged. Harnesses: `tests/conformance/gcp-sdk-go-sql-lro/`, `gcp-sdk-go-fn-lro/`. Remaining P3: auth/endpoint-override docs + the Terraform `apply→clean-plan` round-trip gate (#9).

**P1.2 progress — Pub/Sub console↔SDK shared state (DONE + verified):** the simulator's Pub/Sub handlers (list/create/get/delete topics & subscriptions, publish, pull, ack, topic→subs, console count) now **delegate to the Pub/Sub emulator** via `core/gcp_pubsub_emulator.py` (added `google-cloud-pubsub` to the image), namespaced by the active space's project — so the console and any external gRPC SDK targeting the same project share **one** state. Verified bidirectionally: a topic created via the sim REST API is listed by the external Go SDK, and a topic created by the SDK appears in the sim's list (`tests/conformance/gcp-sdk-go-pubsub-shared/`). Falls back to the in-proc broker when no emulator is configured. **P1.2 Firestore delegation (DONE + verified):** `core/gcp_firestore_emulator.py` wraps the Python `google-cloud-firestore` client (pointed at `FIRESTORE_EMULATOR_HOST`) with a Firestore REST typed-value↔Python converter and nested-subcollection path refs. The doc CRUD + list leaf handlers and the console Firestore count now delegate to the emulator. Verified bidirectionally: a doc written via the sim REST API is read by the external Go Firestore SDK, and an SDK-written doc is read via sim REST (`tests/conformance/gcp-sdk-go-firestore-shared/`). `runQuery/indexes` stay on the SQLite engine (the emulator serves queries natively for external SDKs). **P1.2 COMPLETE** — both Pub/Sub and Firestore share one state between the console and external gRPC SDKs.

**P2 progress — update/patch verbs (the highest-value API CRUD gaps for Terraform/SDK):** added + verified — Compute `instances.setMetadata / setTags` (return a `compute#operation`), Storage `buckets.patch` (PATCH/PUT `/storage/v1/b/{bucket}` — `storageClass/labels/versioning/lifecycle...`), Cloud SQL `instances.patch` (PATCH/PUT → `sql#operation UPDATE`). All verified live. Then added + verified: **IAM service-account keys** (create returns `privateKeyData`, list hides it, delete), **Cloud SQL users + databases**

(list/create/delete → `sql#operation`), and a **console project-IAM-policy editor** (Add/Remove role binding → `setIamPolicy`). Remaining P2 (lower value, deferred): IAM `signBlob` /token minting, API Gateway config/gateway PATCH, VPC routes/router, and the console row-Edit UIs for Compute/Storage instances.

**P3 — Terraform round-trip gate (PASSED , the north-star)**: the real `hashicorp/google` provider (custom endpoints + fake OAuth token) runs `terraform apply` → creates `google_storage_bucket` + `google_pubsub_topic` against the simulator, then `terraform plan` reports **"No changes. Your infrastructure matches the configuration."** (zero drift), and `terraform destroy` cleans up (2 destroyed). Two bugs found & fixed: (1) the provider creates topics via **PUT** (`api_gcp_pubsub_put_topic`) which wasn't delegated to the emulator while GET was → infinite create/read retry loop; now PUT delegates too. (2) the topic view fabricated a default `messageRetentionDuration`: `"604800s"` (a *subscription* default, not a topic one) → `plan` drift; now only emitted when explicitly set. Config: `tests/conformance/terraform-gate/`. This exercises LRO polling + response-shape fidelity end-to-end and is the strongest evidence for the export-to-real-cloud north-star.

**One-line verdict**: Console and **REST-contract** parity are strong (the simulator's own console and raw REST work well across 9 services, with real data planes behind SQL/Storage/Functions/Compute/VPC). The big gaps are **(a) real client-library/CLI verification is absent** (everything is HTTP-tested, not SDK-tested), and **(b) gRPC-default services (Pub/Sub, Firestore) cannot be reached by their official SDKs at all**.

## 2. Console parity (per service)

All 9 services use the same `GCPListShell` (left resource grid + right tabbed detail pane + breadcrumb header). Sidebar

`GCP_CONSOLE_SERVICES`: Compute Engine, Cloud Storage, Cloud SQL, Firestore, Pub/Sub, API Gateway, Cloud Functions, VPC Network, IAM. A Console Home dashboard tiles all 9 with live counts.

| Service     | Resources / tabs                            | Create | Edit                | Delete                 | Extra actions                               | Detail tabs                                   | Console gaps                                                                                                              |
|-------------|---------------------------------------------|--------|---------------------|------------------------|---------------------------------------------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| Compute     | instances, groups, disks, snapshots, images | ✓      | ✗                   | ✓                      | Start/Stop/Reset/SSH                        | overview, networking, security, metadata      | No instance Edit (machine type/tags/labels); groups/disks are list+delete only; no templates/autoscalers/LB/health-checks |
| Storage     | buckets, folders, transfers, IAM&access     | ✓      | bucket ✗ / policy ✓ | ✓                      | Upload object; Edit+Delete policy           | overview, objects, permissions, configuration | No bucket Edit (class/versioning/lifecycle); lifecycle/notifications/encryption not editable                              |
| Cloud SQL   | instances, backups, query insights          | ✓      | ✗                   | ✓                      | Start/Stop/Restart; connection string shown | overview, connectivity, security, operations  | No instance Edit (tier/flags); no users/databases/replicas/SSL UI                                                         |
| Pub/Sub     | topics, subscriptions, schemas              | ✓      | ✓ (topic+sub)       | ✓                      | Publish, Pull, Ack, message introspection   | overview + subs/delivery/definition           | No dead-letter/snapshot/seek UI                                                                                           |
| Firestore   | data, query builder, indexes                | ✓      | ✓ (doc fields)      | ✓                      | Run query (operators), subcollection paths  | data, query                                   | No transactions/batch UI; index hints local-only                                                                          |
| Functions   | functions                                   | ✓      | ✓                   | ✓                      | <b>Invoke</b> (real exec)                   | single (metadata + ops)                       | gen2/triggers/secrets not modeled                                                                                         |
| API Gateway | apis, configs, gateways                     | ✓      | ✗                   | ✓ (config via gateway) | request routing (invoke)                    | per-resource linked tables                    | No Edit; config delete only when attached                                                                                 |
| VPC         | networks, subnetworks, firewalls            | ✓      | firewall ✓          | ✓                      | firewall enforcement (iptables)             | metadata + JSON + counts                      | Subnetwork delete is UI-sim-only; no routers/NAT/peering/routes                                                           |
| IAM         | service accounts (+ project policy)         | ✓      | ✓ (SA)              | ✓                      | policy view                                 | metadata + policy                             | Project-policy editor is view-only; SA keys absent                                                                        |

**Console takeaways:** Every grid now has **Create + Delete**; **Edit** exists for Pub/Sub (topic & sub), Firestore docs, Functions, Firewalls, Service accounts, Storage policy. The remaining console gaps are **row-level Edit for Compute instances**, **Storage buckets**, **Cloud SQL instances**, and **API Gateway**, plus a **project-IAM-policy editor**.

### 3. SDK parity (Java / Go) — the structural gap

This is the weakest dimension and deserves the most attention.

#### 3.1 What exists today

- `core/tooling_simulators.py::sdk_snippet()` returns **static bootstrap snippets only** — e.g. Java `StorageOptions.newBuilder().setHost("http://127.0.0.1:9000")` and Go `storage.NewClient(ctx, option.WithEndpoint(...), option.WithoutAuthentication())`.
- `core/provider_registry.py` declares Java/Go SDK status `partial` with the honest note: **"client wrappers are still missing."**
- No test points a real `com.google.cloud:*` (Java) or `cloud.google.com/go` (Go) client at the simulator.**

#### 3.2 The transport problem (why it's not just "untested")

The official Google client libraries use **two different transports**, and the simulator only offers REST:

| Transport family                               | Services (default)                                                                 | Works against a REST-only sim?                                                                                                                                                                                                                                                       |
|------------------------------------------------|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| REST / HTTP-JSON                               | Cloud Storage, Compute, Cloud SQL Admin, Cloud Functions (admin), API Gateway, IAM | 🟡 <b>Plausibly yes</b> with <code>setHost / WithEndpoint + WithoutAuthentication</code> — <i>if</i> request/response shapes, <code>selfLink</code> s, and long-running-operation polling match. Untested.                                                                            |
| gRPC (default, no REST fallback in common use) | Pub/Sub, Firestore, Datastore, Spanner, Bigtable                                   | 🔴 <b>No.</b> A REST endpoint cannot satisfy a gRPC client. The emulator-host envs ( <code>PUBSUB_EMULATOR_HOST</code> , <code>FIRESTORE_EMULATOR_HOST</code> ) make the libs talk to a local emulator but <b>expect the emulator's gRPC protocol</b> , which the sim does not speak. |

**Cloud SQL is the exception that already works at the SDK level:** the data plane is a **real Postgres/MySQL container**, so JDBC, `database/sql + pq / go-sql-driver`, `psycopg2`, etc., connect over the genuine wire protocol. This is full SDK/driver parity for the *data* plane (not the Admin API).

#### 3.3 Net SDK position

- Best REST candidates** (Storage, Compute, Functions admin, IAM, API Gateway, SQL Admin): one verification pass from "🟡 plausible" → "🟢/🟡 verified."
- Pub/Sub & Firestore:** real SDKs are **blocked** until a gRPC front-end exists (or the official Google emulators are run behind the sim — see §6).
- Storage** is uniquely promising because it's REST **and** backed by `fake-gcs-server` (the official libs even have a `STORAGE_EMULATOR_HOST` mode).

### 4. Utilities parity (gcloud / gsutil / data-plane CLIs)

- gcloud / gsutil translators** (`gcp_gcloud_resolve`, `gcp_gsutil_resolve`) are **command-shape doc helpers** — they parse a command string and return `{service, method, REST route, notes}` for display. They do **not** execute anything or wire a real CLI. Coverage: `compute/storage/sql/pubsub/vpc/iam` command *forms* only.
- Real gcloud** can in principle be pointed at the sim via `CLOUDSDK_API_ENDPOINT_OVERRIDES_<api>=<url>`, and **real gsutil / gcloud storage** at `fake-gcs-server` — but neither is wired, documented, or tested. → 🟡 translator-only.
- Data-plane CLIs are real:** `psql / mysql` connect to the published Cloud SQL ports (5432/3306) against the per-instance DB. This is genuine utility parity for Cloud SQL data.

### 5. API (REST) parity — per-service gaps

Native REST routes are present and the **contract is verified via the HTTP harness**. Remaining per-service gaps vs the real Google APIs:

- **Compute Engine** (🟡 70%): have LCD + start/stop/reset + zone operation GET. **Missing:** `instances.patch / setMetadata / setTags / setMachineType`, `attachDisk / detachDisk`, `disks.resize`, `public images.list / machineTypes.list`, instance templates, autoscalers, MIG operations, accurate LRO `done` polling.
- **Cloud Storage** (🟡 70%): bucket/object LCD, **real media upload + ?alt=media download**, folders (sim-only), transfers, IAM. **Missing:** `buckets.patch`, `objects.compose / rewrite / copy`, resumable uploads, versioning, lifecycle rules, signed URLs, notifications, HMAC keys.
- **Cloud SQL** (🟡 60%, on `v1beta4`): instance LCD + restart, backups, insights (sim-only). **Missing:** `instances.patch` (tier/flags/maintenance), `users.*`, `databases.*`, SSL certs, failover, clone, read replicas, import/export.
- **Pub/Sub** (🟢 85%): topics+subs LCRUD, publish/pull/ack/modifyAckDeadline/purge, schemas, **lease+redelivery, ordering keys, retention/TTL**. **Missing:** dead-letter policy, snapshots + `seek`, BigQuery/GCS subscriptions, detach.
- **Firestore** (🟡 70%): document CRUD with **path-based subcollections**, `runQuery` (operators + composite AND + orderBy), indexes — real SQLite engine. **Missing:** transactions ( `beginTransaction / commit / rollback` ), `batchGet / batchWrite`, `listen` streaming, aggregation queries ( `count / sum / avg` ), backups/PITR.
- **Cloud Functions** (🟡 65%): functions LCRU + `:call` **real subprocess execution** + IAM policy + versions/invocations (sim-only). **Missing:** source-upload-URL deploy flow, event-trigger config, gen2/Cloud Run model, secrets, build steps.
- **API Gateway** (🟡 50%): apis/configs/gateways **LCD only** + custom `:invoke` routing. **Missing:** PATCH for configs/gateways, OpenAPI validation, auth schemes, IAM, managed-service lifecycle.
- **VPC** (🟡 55%): networks LCD, subnetworks LC, firewalls LCURD + **real iptables reconcile**. **Missing:** subnetwork GET/DELETE (real), routes, Cloud Router, Cloud NAT, peering, VPN, firewall-policy hierarchy.
- **IAM** (🟡 55%): project `getIamPolicy / setIamPolicy / testIamPermissions`, service accounts LCRUD+patch, **real opt-in PDP enforcement**. **Missing:** SA keys ( `create / list / delete` ), `signBlob / signJwt`, `generateAccessToken / generateIdToken`, custom roles, org/folder policies, workload identity.

## 6. Cross-cutting gaps

1. **No gRPC transport** → Pub/Sub & Firestore (and any future Spanner/Bigtable/Datastore) official SDKs are unreachable. *Highest-leverage SDK gap.*
2. **No real-client conformance for GCP** → all SDK/CLI parity is unverified (vs AWS's real `boto3` suite). We cannot currently *claim* Java/Go/gcloud parity with evidence.
3. **Auth** — the sim accepts fake/absent tokens; real clients must be told `WithoutAuthentication` / emulator mode. Fine for local use, but it's a deliberate deviation to document.
4. **Long-running operations (LRO)** — Compute/SQL/Functions return operation objects; SDKs and Terraform poll them to `done`. Fidelity here is partial and matters for SDK/Terraform success.
5. **Terraform round-trip gate (pending task #9)** — the north-star "apply → clean plan → export to real cloud" gate isn't in place; it would exercise selfLinks, LROs, and field masks across all services at once.
6. **Pagination tokens, ETags, field masks, quota-project resolution** — partial across services.

## 7. Prioritized recommendations

**P0 — Make parity *provable* (convert theoretical → verified):**

1. Add a **real-SDK GCP conformance harness** mirroring the `boto3` approach: run `google-cloud-storage` (Java + Go) with `setHost / WithEndpoint + WithoutAuthentication` against the sim, and `gcloud / gsutil` with `CLOUDSDK_API_ENDPOINT_OVERRIDES / STORAGE_EMULATOR_HOST`. Start with **Storage, Compute, Cloud SQL Admin, Functions, IAM** (all REST). This turns the 🟡/🟠 SDK cells into evidence-backed ratings and will surface shape mismatches.

**P1 — Unblock the gRPC services (fidelity-principle aligned):** 2. Front Pub/Sub and Firestore with the **official Google emulators** (Pub/Sub emulator, Firestore emulator) as drop-in OSS backends — exactly the pattern already used for Postgres/MySQL (SQL) and fake-gcs-server (Storage). Then `PUBSUB_EMULATOR_HOST / FIRESTORE_EMULATOR_HOST` make the real SDKs work, while the console keeps reading the simulator state. Alternatively add a thin gRPC↔REST shim.

**P2 — Close the highest-value API/console CRUD gaps:** 3. API: `instances.patch / setMetadata / setTags` (Compute), `buckets.patch` (Storage), `instances.patch + users` (SQL), config/gateway PATCH (API Gateway), routes/router (VPC), SA keys + token minting (IAM). 4. Console: row-level **Edit** for Compute instances, Storage buckets, Cloud SQL instances, API Gateway; a **project-IAM-policy editor**.

**P3 — Productionize fidelity:** 5. Document the auth/endpoint-override story per tool; improve LRO polling fidelity; land the **Terraform round-trip gate (#9)**; add `gcloud / gsutil / bq` endpoint-override docs + smoke tests.

## Appendix — evidence basis

- API route inventory: `providers/gcp_routes.py`, `providers/gcp_compute_routes.py` (handlers in `gcp_services.py`, `gcp_storage_sql_vpc.py`, `gcp_iam.py`, `server.py`).
- Console inventory: `static/index.html` (GCP\*View components, `GCP_CONSOLE_SERVICES`).
- Tooling/SDK: `core/tooling_simulators.py`, `core/provider_registry.py`, `providers/capabilities.py`.
- Verified client testing: `tests/conformance/run_conformance.py` (GCP = raw HTTP; AWS = real boto3).
- Data planes: `core/gcp_sql_engine.py` (real DB), `core/gcp_gcs_store.py` (fake-gcs-server), `core/gcp_function_runtime.py` (subprocess), `core/gcp_vpc_enforce.py` (iptables), `core/gcp_iam_policy.py` (PDP).